

A person is running away from the camera on a paved path that leads towards a sunset. The path has a yellow line down the center. The sky is orange and yellow, and there are some trees and a street lamp in the background.

Motivation is what gets
you started. Habit is
what keeps you going.”

My Introduction

- Harshal Garg 🧑
- SDE 2 at Kotak Mahindra Bank 📁
- Ex-Amazon 🛒 | Ex-Oracle 🏛️
- B.Tech in CSE, NIT Jalandhar (2018) 🎓
- 3+ years of teaching experience with 350+ DSA lectures 🧑📖
- Originally from Agra 🕌, currently in Hyderabad 🏙️

Intermediate Module Overview

- Intermediate DSA: Time & Space Complexity
- Intermediate DSA: Introduction to Arrays
- Intermediate DSA: Lab Session on TC, SC, Output & Debugging
- Intermediate DSA: Arrays - Prefix Sum & Carry Forward
- Intermediate DSA: Lab Session on Prefix Sum & Carry Forward
- Intermediate DSA: Arrays: Sliding Window & Contribution Technique
- Intermediate DSA: Lab Session on Memory Management & Sorting Basics
- Intermediate DSA: Bit Manipulations Basics
- Intermediate DSA: Lab Session on 2D Matrices
- Intermediate DSA: Strings [Including String Immutability]
- Intermediate DSA: Lab Session on 2D Matrices & Strings - 2
- Introductory DSA Contest

Class Days

Monday

Wednesday

Friday

7:00 AM IST

We begin every day
at sharp 7:05

7:05 AM

8:20

9:30

Doubts

Note:

- 1. In Intermediate, we shall be learning the concepts around different topics and how to work with certain data structures.
 - This module is dedicated to make you comfortable with Programming.
- 2. Contest will be organised after Intermediate Module.
 - It'll will be for 1.5 hours and will be conducted within class duration followed by Contest Discussion (Instructor shall be discussing contest problems).
 - It'll consist of 4 questions and we expect you to solve ≥ 3 problems. If for any reason you are unable to solve, then we shall also be having re-attempts as well. (We'll provide more info on re-attempts moving forward)
 - Contests are critical to retaining what you have learnt and measuring where you need improvement. Please take contests seriously.
- 3. Be consistent in solving problems. If stuck, please post the issue in your WA/Slack group and let's make it a habit of helping each other as it will eventually help you to be better.

FAQs :

- Notes will be uploaded after the class.
- Assignments will be unlocked after the class ends.
- There is no deadline for assignments.
- If asking a question, ask in public chat.
- If answering a question, answer in private chat.

$$\frac{PSP}{\frac{Problems\ Solved}{Problems\ Unkown}} \times 100\%$$

Today's Agenda:-

- 1. Count of Factors
- 2. Basic Maths
- 3. Situations
- 4. Comparing 2 algorithms
- 5. Big O — Time Complexity .

Q.1. Given N , return the count of factors of N .

Factor :- i is a factor of N .
 $\rightarrow i$ completely divides N .
 $N \% i == 0$

Q.1 $N = 24$
1, 2, 3, 4, 6, 8, 12, 24
8.

Q.2 $N = 10$
1, 2, 5, 10
4.

Brute Force Idea:-

Iterate from 1 to N & check if i is a factor of N . If yes, increment the count.

```
int CountFactors (int N) {  
    int count = 0;  
    for (i  $\rightarrow$  1 to N) {  
        if (N % i == 0) {  
            count++;  
        }  
    }  
    return count;  
}
```

$\Theta(N)$

Analysing execution time of the code

Ass'n :- A server executes 10^8 iterations per second.

$$\frac{1}{10^8} \times 10^{18}$$

N

Iterations

Execution Time

10^8

10^8

1 sec.

10^9

10^9

10 sec.

10^{18}

10^{18}

10^{10} sec.

≈ 317 years.

Optimisation

$i \rightarrow N/i$ is also a factor of N .

$N=24$

$N=100$

i		N/i
1	<	24
2	<	12
3	<	8
4	<	6
6	>	4
8	>	3
12	>	2
24	>	1

i		N/i
1	<	100
2	<	50
4	<	25
5	<	20
10	<	10
20	>	5
25	>	4
50	>	2
100	>	1

Idea:- Iterate for all $i \leq N/i$ & count the factors in pairs.

~~X~~ Missing For Perfect Square.

int countFactors (int N) {

int count = 0;

for (i = 1; i ≤ N/i; i++) {

if (N % i == 0) {

count += 2;

}

}

return count;

3

$N=24$

$i = 1 \times 2 \times 3 \times 4 \times 5$

Count = ~~1~~ ~~2~~ ~~4~~ ~~8~~

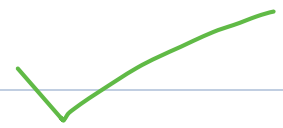
$N=100$

$i = 1; 2 \times 3 \times 4 \times 5 \times 6 \times 7 \times 8 \times 10$

Count = ~~1~~ ~~2~~ ~~4~~ ~~8~~ ~~10~~

$$i \leq N/i \rightarrow i^2 \leq N ; i \leq \sqrt{N}$$

Code



```

int CountFactors (int N) {
    int Count = 0;
    for (i=1; i*i <= N; i++) {
        if (N%i == 0) {
            if (i == N/i) {
                Count += 1;
            } else {
                Count += 2;
            }
        }
    }
    return Count;
}

```

Takes care of P.S.

3.1.4

$O(\sqrt{N})$

N

Iterations

Execution Time

10^8

10^4

$\frac{1}{10^8} \times 10^4 = 0.0001 \text{ s}$

10^9

$10^4 \times \sqrt{10}$

$\sqrt{10} \times 0.0001 \text{ s}$
 $= 0.000314 \text{ s}$

10^{18}

$\sqrt{10^{18}} = 10^9$

$\frac{1}{10^8} \times 10^9 = 10 \text{ sec.}$

317 years

$\longrightarrow 10 \text{ sec.}$

Prime Numbers

A no. having exactly 2 factors.

Q:-

10, 11, 23, 2, 25, 27, 31

4 2 2 2 3 4 2

ans = 4.

Q:- Given N, return true if the no. is Prime else return false.

Call CountFactors.

If return 2 → true
else

false.

Code

boolean CheckPrime (int N) {

int factors = CountFactors(N);

return (factors == 2);

}

Basic Maths

$[\rightarrow$ include
 $(\rightarrow$ exclude.

1. $[a, b] \rightarrow$ All no. within the range
 $a \rightarrow b$ including both a & b

Q: 4 $[3, 10] = 3, 4, 5, 6, 7, 8, 9, 10$
8 = $10 - 3 + 1$

Q: 5. $[a, b] = b - a + 1$

2. $(a, b) = b - a - 1$

$(3, 10) = 4, 5, 6, 7, 8, 9$
 $= \underline{6}$

3. $[a, b) \text{ or } (a, b]$
 $= \underline{\underline{b - a}}$

Sum of first N natural Numbers

Gauss:

Q:-6

$$S = 1 + 2 + 3 + 4 + \dots + 100$$

$$+ S = 100 + 99 + 98 + 97 + \dots + 1$$

$$2S = 101 + 101 + 101 + 101 + \dots + 101$$

$$= 101 \times 100$$

$$S = \frac{101 \times 100}{2} = 5050$$

$\therefore$ For any N :
Sum of first N natural numbers

$$= \frac{(N+1) \times N}{2}$$

1, 2, 3, ..., N

Geometric Progression :- A series of numbers having a common ratio b/w them.

ex: 5, 10, 20, 40, 80, 160, ...

$$\frac{10}{5} = \frac{20}{10} = \frac{40}{20} = \frac{80}{40} = \dots \textcircled{2}$$

Sum of first N no's in a G.P.

$$S_N = \frac{a(r^N - 1)}{r - 1}$$

$$\underline{\underline{r \neq 1}}$$

$a \rightarrow$ first term of G.P.

$r \rightarrow$ common ratio

$N \rightarrow$ No. of terms

2, 4, 8, 16, 32

$$S_5 = \frac{a(r^N - 1)}{r - 1} = \frac{2(2^5 - 1)}{2 - 1}$$

$$= 2 \cdot (2^5 - 1) = \underline{\underline{62}}$$

Iteration :- No. of times a loop is getting executed:

Q:- 7.

```
for (i → 1 to N) {  
    if (i == N) { break; }  
}
```

$N = 5$

$i = 1, 2, 3, 4, 5$

(N)

↑
true..

Q:- 8.

```
for (i → 0 to 100) {  
    s = s + i + i^2;  
}
```

$$[0, 100] = 100 - 0 + 1 = \underline{\underline{101}}$$

Q:-9

```
func(){  
    for(i -> 1 to N){  
        if(i % 2 == 0){  
            print(i);  
        }  
    }  
    for(j -> 1 to M){  
        if(j % 2 == 0){  
            print(j);  
        }  
    }  
}
```

$$C1, N] = \underline{N}$$

$$C1, m] = m$$

$$(N+m).$$

8:35

Comparing 2 algos

Romesh

15 sec
(Windows XP)

↓ mac2
7 sec.
(C++)

Suresh

10 sec.
(Mac2)
(Python)

↓ C++
5 sec.

Conclusion: Execution time is not the right criteria to compare 2 algos as it depends on various factors like OS, H.W, language etc... -

∴ The correct criteria to compare 2 algos is No. of iterations.

Ramesh's

$$100 \times \log N$$

Suresh

$$N/10$$

Both graphs met at around 3500.

For $N < 3500$; Suresh's algo is better

& for $N > 3500$; Romesh's algo
is better.

$\therefore$ Romesh's algo is better

Asymptotic Analysis of Algorithms $\rightarrow$ Big O.

Analysing algorithms for larger inputs.

Big O.

Steps to Calculate Big O:

1. Cal. the no. of iterations of your code.

2. Remove the low order terms.

3. Remove the constant coefficient

Comparsion Order:

1 <

$$\log(N) < \sqrt{N} < N < N \log(N) < N \sqrt{N} < N^2 < N^3 < 2^N < N! < N^N$$

Using an example

$$N = 36$$

$$5 < 6 < 36 < 36 \cdot 5 < 36 \cdot 6 < 36^2 < 36^3 < 2^{36} < 36! < 36^{36}$$

Examples:

$$\textcircled{1} \quad 4N^2 + 3N + 1 = \underline{\underline{O(N^2)}}$$

$$\textcircled{2} \quad 4N^2 + 3N + 6\sqrt{N} + 5N + 8 = \underline{\underline{O(N^2)}}$$

Q:-10

$$4N + 3N \log(N) + 1 = O(N \cdot \log N)$$

Q:-11

$$4N \log N + 3N \sqrt{N} + 10^6 = \underline{\underline{O(N \cdot \sqrt{N})}}$$

Why neglecting Lower Order Terms ?

Let's say my code has total
iterations as $N^2 + 10N$

$$L.O.T = 10N$$

Percentage Contribution
of L.O.T. in total iterations

$$= \frac{10N}{N^2 + 10N} \times 100$$

N	Total Iterations	Iterations due to L.O.T.	Percentage Contribution
	$N^2 + 10N$	$10N$	
10	200	100	50 %
100	$10^4 + 10^3 = 11000$	10^3	9 %
1000	$10^6 + 10^5$	10^5	0.1 %

∴ As we increase the value of N,
the percentage contribution of L.O.T
reduces drastically & thus it
can be ignored.

Why Neglecting Constant Coefficients?

Algo 1	Algo 2	Winner
$100 \log N$	$N / 10$	<u>Algo 1.</u>
$1000 \log N$	$10 N$	<u>Algo 1.</u>

Issues with Big(O) :

Issue 1: Taking a look at Big O, we cannot determine which algo is better for smaller inputs.

$$\begin{array}{ll} \text{Algo 1} \rightarrow 10^3 N & \rightarrow O(N) \\ \text{Algo 2} \rightarrow N^2 & \rightarrow O(N^2) \end{array}$$

N	Algo 1	Algo 2	Winner
10	10^4	10^2	Algo 2:
1000	10^6	10^6	Equal.
<u>1001</u>	$10^3(10^3+1)$	$(10^3+1)(10^3+1)$	<u>Algo 1.</u>

Issue 2: Two algos can have the same Big O but still one algo may be better than the other one.

ex:- Write a function that prints all even no's from 1 to N.

Algo 1

for ($i=1$; $i \leq N$; $i++$) {

if ($i \% 2 == 0$) {
print(i);
}

}

$O(N)$.

Algo 2

for ($i=2$; $i \leq N$; $i+=2$) {

print(i);
}

$$\frac{N}{2}$$

$$\rightarrow \underline{O(N)}$$

$\ll$ 4.2



2 consecutive
classes.